EcoTayo

A Technology-Driven Approach to Recyclable Waste Management

Technical Manual

A.Y. 2025-2026, Term 1

Aniago, Ople, Sioco, Vidal

Contents

System Set-up for Developer Mode **1**
Prerequisites 1
1. Clone the GitHub Repository 1
2. Install Node.js Dependencies 2
3. Environment Configuration 3
3.1 Create Backend Environment File 3
3.2 Create Frontend Environment File 5
4. Firebase Configuration 6
4.1. Firebase Project Setup 6
4.2. Get Firebase Configuration 6
4.3 Enable Required Firebase Services 7
5. Google API Configuration 9
5.1 Create Google Cloud Project 9
5.2 Create Service Account Key 9
5.3. Add Service Account Key to Project 10
5.4. Create OAuth 2.0 Credentials 10
Running the System in Development Mode **12**
6. Start the Servers 12
6.1 Start Both Services Concurrently (Recommended) 12
6.2 Verify the System is Running 12
6.3. Note for Firebase Queries 12
Project File Structure **14**
Troubleshooting **16**
Issue: "Cannot find module" errors 17
Issue: Firebase initialization errors 17
Issue: CORS errors 17
Issue: Port already in use 17

System Set-up for Developer Mode

Prerequisites

Before setting up the EcoTayo system for development, ensure you have the following software installed on your machine:

- **Git**: For cloning the repository and version control
- **Node.js**: JavaScript runtime (version 16.0.0 or higher recommended)
- **npm**: Node Package Manager (version 8.0.0 or higher) - typically comes with Node.js
- **Visual Studio Code** (or any preferred code editor)

1. Clone the GitHub Repository

The EcoTayo project is stored in a GitHub repository. To run the system locally, clone the repository using the following command:

```shell
# Clone the repository
git clone
https://github.com/megan-ryleneryl/CAPSTONE-Waste-Recycling.git

# Switch to the main branch
cd CAPSTONE-Waste-Recycling
git checkout main
```

Then make sure to work in the correct project directory:

```shell
# If not in the directory already
cd CAPSTONE-Waste-Recycling
```

2. Install Node.js Dependencies

The project uses **npm** as the package manager. Install all required dependencies for both the backend and frontend:

```shell
# Install dependencies for the entire project
# In the main directory
npm install

# Navigate to the client directory and do the same
cd client
npm install

# Once done, move back to the main directory
cd ..
```

These commands will install all packages listed in the root **package.json** and those in the **client/package.json** file.

To verify that Node.js and npm are installed correctly, run:

```shell
node --version
npm --version
```

3. Environment Configuration

3.1 Create Backend Environment File

Create a **.env** file in the root directory of the project. This file contains configuration for both the backend and frontend services.

Create the file in the root directory:

On Windows (PowerShell):

```shell
New-Item -Path .env -ItemType File
```

On Windows (Command Prompt):

```shell
type nul > .env
```

On macOS/Linux:

```shell
touch .env
```

Add the following environment variables to the .env file:

```
None
# Firebase Configuration
FIREBASE_API_KEY=
FIREBASE_AUTH_DOMAIN=
FIREBASE_PROJECT_ID=
FIREBASE_STORAGE_BUCKET=
FIREBASE_MESSAGING_SENDER_ID=
FIREBASE_APP_ID=

# Server Configuration
NODE_ENV=development
PORT=3001
CLIENT_URL=http://localhost:3000

# Rate Limiting (Relaxed for development)
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX_REQUESTS=1000

# CORS Settings (Allow local development ports)
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:3001,http://local
host:8080,http://127.0.0.1:3000

# Team Development Settings
ENABLE_DEBUG_ROUTES=true

# Google Secrets
GOOGLE_APPLICATION_CREDENTIALS=./config/serviceAccountKey.json
GOOGLE_CLIENT_ID=
```

3.2 Create Frontend Environment File

Create a separate **.env** file specifically for the frontend service. Navigate to the client folder and perform the same steps. Create the file in the client directory (no need to change directories).

On Windows (PowerShell):

```shell
New-Item -Path client/.env -ItemType File
```

On Windows (Command Prompt):

```shell
type nul > client\.env
```

On macOS/Linux:

```shell
touch client/.env
```

Add the following environment variables to the .env file:

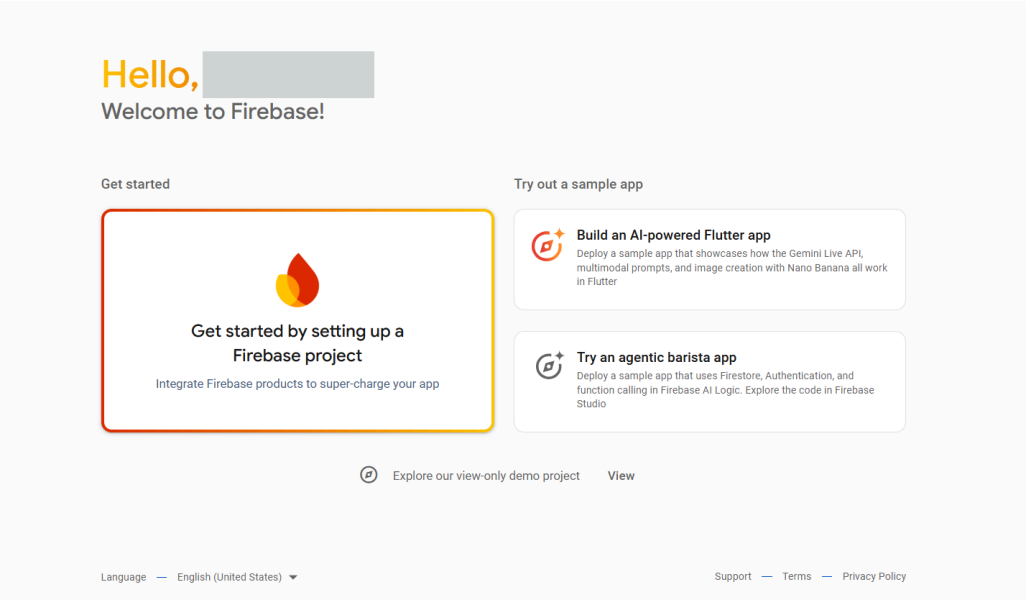
```none
# Google API Credentials
REACT_APP_GOOGLE_CLIENT_ID=
REACT_APP_API_URL=http://localhost:3001/api
```

4. Firebase Configuration

Firebase is used for authentication, database, and storage in the EcoTayo system.

4.1. Firebase Project Setup

1. Go to [Firebase Console]
2. Click **"Add Project"** or select an existing project
3. Name the project (e.g., "EcoTayo Recycling System")
4. Accept the Firebase terms, and proceed through the menus. You may personally choose to reject or accept the feature offers.
5. Then after the last menu, click **"Continue"** and wait for it to complete



The project initialization, step 1

4.2. Get Firebase Configuration

Once your Firebase project is created:

1. Click the **"Settings"** icon (gear) in the top left
2. Select **"Project Settings"**
3. Under **"General"**, scroll down to the **"Your Apps"** section

4. If no app exists, add an app by selecting **Web** (this symbol: </>)
    a. Name the web app (e.g. "EcoTayo Recycling System") and click **"Register"**. No need to turn on Firebase hosting
    b. Accept the default **"Use npm"** option and click **"Continue to console"**
5. Copy the Firebase configuration object (it will look similar to the sample in **.env**, but in JSON form)
6. Update the FIREBASE_ variables in your backend **.env** file with these values
7. Update the "Your actual Firebase configuration" values in **./config/firebase.js** file with these values.

```
// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyAFgWea0qXj85pwHiZDA-fHCd6bj_awqGs",
  authDomain: "ecotayo-recycling-system.firebaseapp.com",
  projectId: "ecotayo-recycling-system",
  storageBucket: "ecotayo-recycling-system.firebasestorage.app",
  messagingSenderId: "266416074639",
  appId: "1:266416074639:web:452e4991d5dbd4573625fa"
};
```

A sample Firebase configuration

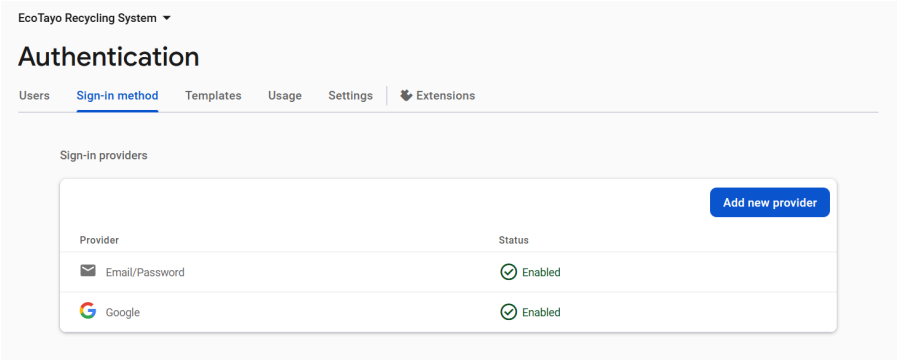
4.3 Enable Required Firebase Services

To access authentication:

1. In the Firebase Console, look at the **left sidebar**
2. Under the "Build" section, click on **"Authentication"** (you may need to scroll down in the sidebar to see it)
3. Look for the button that says **"Get Started"**, click it
4. You are now in the Authentication menu

In the Authentication section:

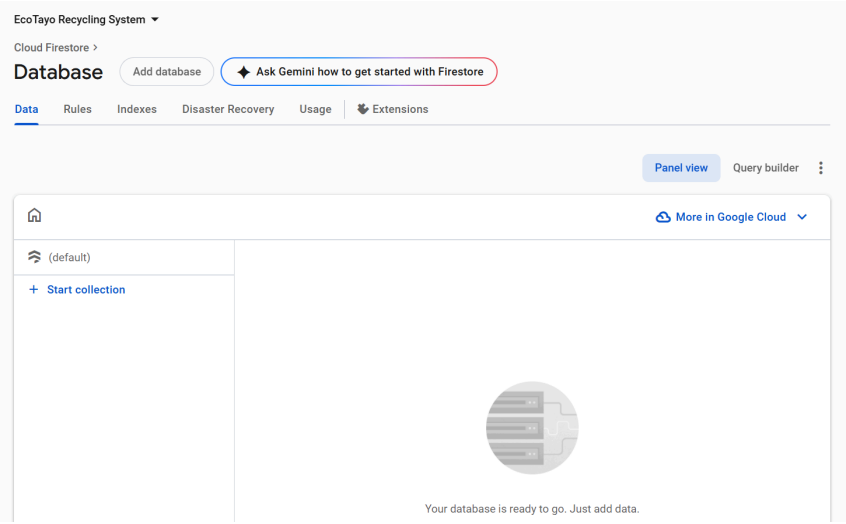
1. Click the **"Sign-in method"** tab at the top
2. Click **"Email/Password"** and enable it (only the regular sign-in one)
3. Click **"Add new provider"**, select **"Google"**, and enable it
4. You can optionally set a public-facing name for the project
5. Set the support email to your own
6. Save your changes



Completed authentication set-up

To access Firebase Database:

1. In the Firebase Console left sidebar, under "Build" section
2. Click **"Firestore Database"**
3. Click **"Create Database"**
4. Select the Standard Edition and click "Next"
5. Select the server region most applicable to you (e.g. Singapore), click "Next"
6. Choose **"Start in test mode"** for development
7. Click **"Create"** and wait for the process to complete



Completed database set-up

5. Google API Configuration

The EcoTayo system uses Google APIs for login services.

5.1 Create Google Cloud Project

1. Go to Google Cloud Console
2. Click the **"Select a project"** dropdown at the top
3. If you completed the previous steps, you may skip the next 3 steps
    a. Click **"New Project"**
    b. Name the project (e.g. EcoTayo Recycling System)
    c. Click **"Create"** and wait for it to complete

5.2 Create Service Account Key

1. Open the upper-left hamburger menu and look for **APIs & Services → Credentials**
2. Click **"Create Credentials" → "Service Account"**
3. Fill in the service account details:
    - Service account name: **ecotayo-backend**

- Click **"Create and Continue"**

4. Grant the role **"Editor"** (for development purposes)
5. Click **"Continue"** then **"Done".**
6. In the service accounts list, click on the created account
7. Go to the **Keys** tab
8. Click **"Add Key"** → **"Create New Key"**
9. Choose **JSON** format
10. Click **"Create"** - the key will download automatically

5.3. Add Service Account Key to Project

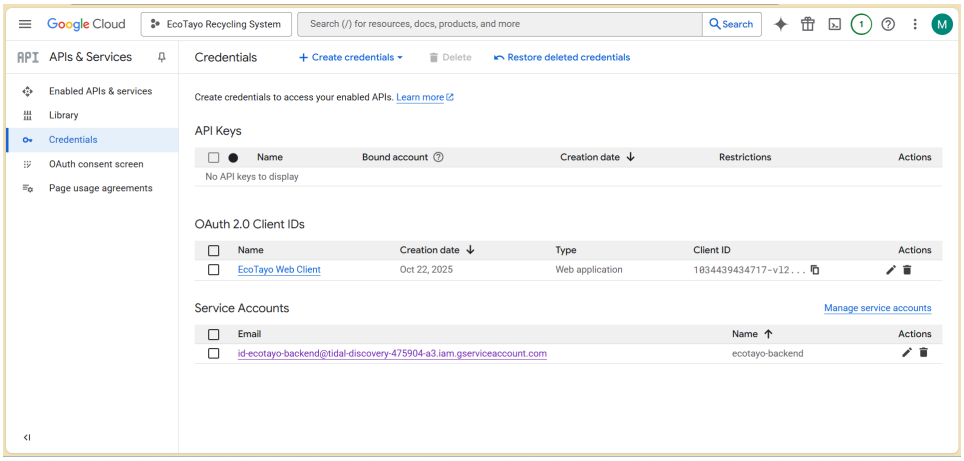
1. Rename the file to **serviceAccountKey.json**
2. Move the downloaded JSON file in your project:
    - Directory Path: **./config/serviceAccountKey.json**

5.4. Create OAuth 2.0 Credentials

1. Open the upper-left hamburger menu and look for **APIs & Services** → **Credentials**
2. Click **"Create Credentials"** → **"OAuth client ID"**
3. If prompted, configure the OAuth consent screen first:
    - Choose **External** user type
    - Fill in required fields with your app information
    - Add test users (your email addresses)
4. Return to the APIs & Services → Credentials and create OAuth Client ID:
    - Application type: **Web application**
    - Name: **EcoTayo Web Client**
    - Add these authorized redirect URIs:
        i. http://localhost:3000
        ii. http://localhost:3001
        iii. http://localhost:5000
5. Scroll down and click **"Create"**
6. Copy the Client ID and add it to the .env in both the backend and frontend:

```
None
# In the backend .env
GOOGLE_CLIENT_ID=your_client_id_here

# In the frontend .env
REACT_APP_GOOGLE_CLIENT_ID=your_client_id_here
```



Final Google Cloud set-up

Running the System in Development Mode

6. Start the Servers

6.1 Start Both Services Concurrently (Recommended)

Since the project is configured with **concurrently**, you can run both services from the root directory:

```shell
npm run dev
```

This will start both the backend (port 3001) and frontend (port 3000) simultaneously.

6.2 Verify the System is Running

1. Open your browser and navigate to: **http://localhost:3000**
2. You should see the EcoTayo landing page
3. Check the browser console for any errors
4. Verify Firebase connection by attempting to sign up/log in

6.3. Note for Firebase Queries

When you create a fresh Firestore database and run the application for the first time, you may see errors in the server console like:

```shell
The query requires an index. You can create it here:
https://console.firebase.google.com/v1/r/project/...
```

This is expected behavior. Firestore requires composite indexes for complex queries. When you see this error:

1. Open the link in your browser
2. Firebase will automatically prompt you to create the required index
3. Click "Create Index" and wait for it to complete (usually 1-2 minutes)
4. Once created, the error should no longer appear

You may need to repeat this process for multiple queries the first time you use the application. This is a one-time setup for each Firestore database.

Project File Structure

```
None
recycling-platform-backend/
|
├── client/                              # Frontend React Application
|   ├── public/
|   |   ├── index.html
|   |   └── manifest.json
|   |
|   ├── src/
|   |   ├── components/                  # Reusable React Components
|   |   |   ├── modal/
|   |   |   |   └── ModalPortal.js
|   |   |   ├── navigation/              # Navigation components
|   |   |   ├── profile/
|   |   |   |   ├── DeleteAccountModal/
|   |   |   |   |   └── DeleteAccountModal.js
|   |   |   |   ├── ApplicationStatusTracker/
|   |   |   |   |   └── ApplicationStatusTracker.js
|   |   |   |   ├── PreferredTimesModal.js
|   |   |   |   └── PreferredLocationsModal.js
|   |   |   └── posts/
|   |   |       └── PostDetails/
|   |   |           ├── PostDetails.js
|   |   |           └── PostDetails.module.css
|   |   |   └── ...
|   |   |
|   |   ├── pages/                       # Page Components
|   |   |   ├── Landing.js         # Each page has a .js and .css
|   |   |   ├── Landing.module.css
|   |   |   ├── Profile.js
|   |   |   ├── Profile.module.css
|   |   |   ├── ...
|   |   |
|   |   ├── context/
|   |   |   └── AuthContext.js
```

```
│   │   │
│   │   ├── services/                    # API Services
│   │   │   └── firebase.js              # Firebase config (client)
│   │   │   └── ...
│   │   │
│   │   ├── styles/                      # CSS Modules
│   │   │   └── variables.css             # Global CSS variables
│   │   │   └── iconSizes.js
│   │   │
│   │   ├── App.js                       # Root Component
│   │   ├── App.css
│   │   └── index.js                      # React Entry Point
│   │   └── index.css
│   │
│   ├── package.json
│   ├── package-lock.json
│   ├── .gitignore
│   └── .env                             # Frontend Environment Variables
│
├── config/                              # Backend Configuration
│   ├── firebase.js                      # Firebase Admin SDK Setup
│   └── serviceAccountKey.json           # Google Service Account (NOT in git)
│
├── models/                              # Firestore Data Models
│   ├── Users.js                         # User model
│   ├── Posts.js                         # Base Post model
│   ├── WastePost.js                     # Waste post model
│   ├── InitiativePost.js                # Initiative post model
│   ├── ForumPost.js                     # Forum post model
│   ├── Message.js                       # Message/Chat model
│   ├── Pickup.js                        # Pickup schedule model
│   ├── Application.js                   # User applications model
│   ├── Notification.js                  # Notification model
│   ├── Point.js                         # Points/rewards model
│   ├── Badge.js                         # Badge model
│   ├── Comment.js                       # Comment model
│   ├── Like.js                          # Like model
│   └── Material.js                      # Recyclable materials model
│
```

```
├── routes/                          # API Route Definitions
│   ├── authRoutes.js                # Authentication routes
│   ├── postRoutes.js                # Post management routes
│   ├── profileRoutes.js             # User profile routes
│   ├── messageRoutes.js             # Messaging routes
│   ├── pickupRoutes.js              # Pickup management routes
│   ├── notificationRoutes.js        # Notification routes
│   ├── analyticsRoutes.js           # Analytics & visualization routes
│   ├── materialRoutes.js            # Material management routes
│   ├── pointRoutes.js               # Points system routes
│   └── psgc.js                      # Philippine location data routes
│
├── controllers/                     # Request Handlers (Business Logic)
│   ├── authController.js            # Auth operations
│   ├── postController.js            # Post operations
│   ├── pickupController.js          # Pickup operations
│   ├── messageController.js         # Message operations
│   └── analyticsController.js       # Analytics operations
│
├── middleware/                      # Express Middleware
│   └── auth.js                      # JWT verification & role checks
│
├── services/                        # Business Logic Services
│   ├── auth-service.js              # Auth operations & JWT
│   ├── storage-service.js           # File upload handling (Multer)
│   ├── notification-service.js      # Notification logic
│   └── geocodingService.js          # Location services
│
├── uploads/                         # Local File Storage
│   ├── profile-pictures/            # User profile pictures
│   ├── posts/                       # Post images
│   └── applications/                # Application documents
│
├── server.js                        # Main Backend Entry Point
├── package.json                     # Backend Dependencies
├── package-lock.json
├── .env                             # Backend Environment Variables
├── .gitignore                       # Git Ignore Rules
└── README.md                        # Project Documentation
```

Troubleshooting

Issue: "Cannot find module" errors
Solution:

```shell
# Clear node_modules and reinstall
rm -rf node_modules client/node_modules
npm install
```

Issue: Firebase initialization errors
Solution:

1. Verify Firebase credentials in **.env**
2. Check that Firebase services are enabled in Firebase Console
3. Ensure **serviceAccountKey.json** exists in **./config/**

Issue: CORS errors
Solution:

1. Verify **CORS_ORIGIN** in **.env** matches your local ports
2. Ensure both services are running on correct ports (frontend: 3000, backend: 3001)

Issue: Port already in use
Solution:

```shell
# Kill process on port 3000
lsof -ti:3000 | xargs kill -9

# Kill process on port 3001
lsof -ti:3001 | xargs kill -9
```

For additional support or questions, contact the development team or check the project's GitHub issues page.